

Payments.lk Payment Gateway Integration

AI-Integrated E-Commerce Platform

Complete Technical Implementation, Security, Testing and Sandbox Payment Flow

Project:	AI-Integrated E-Commerce Platform
Technology Stack:	React 18 (Vite) + Node.js/Express + MongoDB + Payments.lk
Gateway Integration:	Payments.lk Sandbox REST API (v1 Checkouts)
Document Purpose:	University Project Documentation & Technical Report
Security Level:	Sandbox Verified (No Production Credentials Exposed)
Date:	September 2026

2. Table of Contents

This technical report documents the complete implementation, security design, testing, and sandbox payment flows of the Payments.lk Sandbox Payment Gateway.

2. Table of Contents	Page 2
3. Introduction	Page 3
4. Payment System Requirements	Page 3
5. System Architecture	Page 5
6. Technology Stack	Page 6
7. Project File Structure	Page 6
8. Environment Configuration	Page 8
9. Payments.lk Checkout API	Page 8
10. Amount Conversion	Page 10
11. Idempotency	Page 10
12. Hosted Checkout Flow	Page 11
13. Frontend Implementation	Page 11
14. Backend Implementation	Page 11
15. Database / Order Model	Page 12
16. Webhook Architecture	Page 12
17. Webhook Security	Page 12
18. Webhook Events	Page 14
19. Payment Success Flow	Page 14
20. Payment Failure Flow	Page 14
21. Security Design	Page 14
22. ngrok Development Setup	Page 15
23. Docker Setup	Page 15
24. Testing	Page 15
25. Unit Test Coverage	Page 15
26. Real Sandbox API Verification	Page 17
27. Troubleshooting History	Page 17
28. Complete End-to-End Flow	Page 17
29. Demonstration Procedure	Page 17
30. Limitations	Page 18
31. Future Improvements	Page 18
32. Conclusion	Page 18
33. References	Page 18

3. Introduction

Online payment gateway integration is a fundamental capability in modern web application engineering. In an e-commerce ecosystem, offering safe, reliable, and localized checkout channels is essential to establishing customer trust, maintaining transaction integrity, and minimizing drop-off during final purchasing.

For this AI-Integrated E-Commerce Platform, Payments.lk was selected as a primary Sri Lankan sandbox payment gateway due to its modern RESTful Checkout API architecture, hosted checkout page capabilities, and support for automated, cryptographically signed webhook notifications.

Payments.lk provides a comprehensive sandbox environment that enables developers to simulate checkout session generation, customer browser redirection, sandbox credit card payment execution, and HMAC signature verification without processing real monetary transactions.

Crucially, this architecture enforces a strict separation of concerns: the merchant e-commerce platform handles cart pricing, inventory locks, and order record creation, while Payments.lk processes sensitive payment card data on its secure PCI-DSS compliant hosted payment platform.

4. Payment System Requirements

Functional Requirements

- Customer Gateway Selection: Enable authenticated buyers to choose Payments.lk as their payment gateway at checkout.
- Server-Side Order Creation: Compute total purchase amounts server-side from active Cart items and persist a pending Order in MongoDB.
- Checkout Session Generation: Exchange order reference and amount with Payments.lk REST API via POST `/v1/checkouts`.
- Hosted Redirection: Redirect customer browser to the secure hosted Payments.lk checkout URL (`https://payments.lk/checkout/...`).
- Sandbox Payment Simulation: Provide an interactive sandbox checkout experience allowing successful, failed, and cancelled payment testing.
- Webhook Event Ingestion: Accept and process raw HTTP POST notifications sent by Payments.lk to `/api/payments/webhook`.
- Order Status Synchronization: Update database payment status (pending -> paid / failed) upon webhook verification.
- Payment Status Feedback Page: Provide clear status reporting at `https://localhost:5173/payment/status/{orderId}`.
- Idempotency Protection: Prevent duplicate session generation and reuse existing checkout URLs for pending orders.
- Cart Clearing: Automatically empty the user's active shopping cart upon successful payment confirmation.
- Authorization Guard: Restrict access to order payment status queries strictly to the order owner or system admin.
- Audit Logging: Maintain detailed server-side audit logs for diagnostic monitoring without printing sensitive credentials.

Non-Functional Requirements

- Security & Credential Protection: Store secret keys strictly in backend environment variables (`.env`). Never leak credentials to client bundles, logs, or repositories.

- **Server-Side Amount Calculation:** Compute total order amounts exclusively from MongoDB product records to prevent client-side price tampering.
- **Webhook HMAC Signature Verification:** Authenticate all incoming webhooks using HMAC-SHA256 signature checks with timing-safe comparison.
- **Replay Protection:** Track processed webhook event IDs to prevent duplicate processing of replayed events.
- **High Availability & Resilience:** Implement timeout handling and diagnostic fallback logging for network failures.

5. System Architecture

The application follows a decoupled modern architecture comprising a React Single-Page Application (SPA), a Node.js/Express REST API backend, a MongoDB database, and Payments.lk hosted payment infrastructure.

The interaction topology involves 6 distinct architectural entities communicating via HTTPS protocols:

1. React Frontend SPA: Manages UI checkout interactions and status rendering under HTTPS (<https://localhost:5173>).
2. Node.js / Express Backend API: Executes server-side price calculation, Order persistence, API request dispatching, and Webhook verification (Port 8080).
3. MongoDB Database: Stores Order documents, item snapshots, payment statuses, and checkout URLs (Port 27017).
4. Customer Browser: Interacts with merchant frontend and redirects to hosted payment forms.
5. Payments.lk Sandbox API: Issues checkout session URLs and handles card processing (<https://api.payments.lk/v1>).
6. ngrok HTTPS Tunnel: Forwards asynchronous webhook POST notifications from Payments.lk to local backend (localhost:8080).

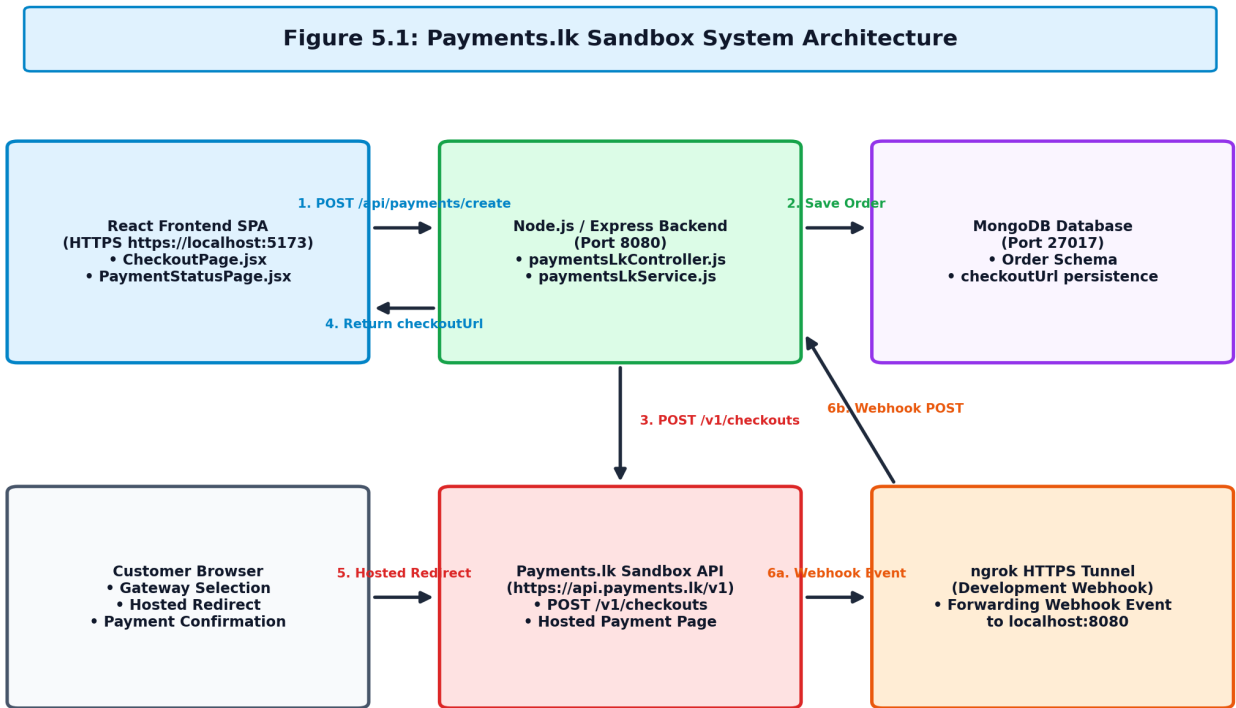


Figure 5.1: Payments.lk Sandbox System Architecture Topology

6. Technology Stack

The payment integration relies strictly on modern, verified technologies present in the workspace codebase:

Component / Layer	Technology Specification	Role & Purpose
Frontend Framework	React 18 SPA built with Vite	User interface, gateway selection, and payment status routing
Frontend SSL Transport	@vitejs/plugin-basic-ssl (HTTPS)	Enforces HTTPS (https://localhost:5173) matching gateway return requirements
Backend Framework	Node.js (v22) + Express REST API	Order management, Payments.lk API integration, and webhook engine
Database & ODM	MongoDB + Mongoose ODM	Persists Order state, payment IDs, checkout URLs, and item snapshots
Payment Gateway API	Payments.lk REST API v1 Checkouts	Generates hosted checkout sessions via POST https://api.payments.lk/v1/checkouts
Webhook Tunneling	ngrok HTTPS Forwarding	Tunnels external webhooks (https://retype-activity-recite.ngrok-free.dev) to local port 8080
Cryptographic Security	Node.js crypto (HMAC-SHA256)	Verifies webhook signatures with timing-safe string comparison (timingSafeEqual)
Containerization	Docker & Docker Compose	Orchestrates backend, frontend, MongoDB, and Redis containers

7. Project File Structure

The payment gateway module is structured cleanly across backend services, Express controllers, routers, database models, unit tests, and frontend React components:

```
Backend/backend-inter/
  ├── controllers/
  │   └── paymentsLkController.js # Express controller (createPaymentSession, handlePaymentsLkWebhook,
  │                               getPaymentStatus)
  ├── services/
  │   └── paymentsLkService.js # Payments.lk API HTTP client, cents formatter & HMAC signature verifier
  ├── router/
  │   └── paymentsLkRouter.js # Express routes (/api/payments/create, /webhook, /status/:orderId)
  ├── models/
  │   └── order.js # Mongoose Order schema with payment status enums & checkoutUrl persistence
  ├── tests/
  │   └── paymentsLkService.test.js # 19/19 Unit test suite for cents conversion, HMAC checks & authorization
  ├── scratch/
  │   └── testRealApiCheckout.js # Live API diagnostic verification script testing POST /v1/checkouts
  ├── index.js # Express application mounting paymentsLkRouter under /api/payments
  └── .env # Server environment secrets & configuration

Frontend/my-react-app/
  ├── src/
  │   └── pages/
  │       └── CheckoutPage.jsx # Checkout interface & gateway selection logic
```

```
■ ■ ■ ■ PaymentStatusPage.jsx # Status display route handling /payment/status/:orderId
■ ■ ■ ■ services/
■ ■ ■ ■ paymentsLkService.js # Client API fetch helper methods (createPaymentsLkCheckout, getPaymentsLkStatus)
```

8. Environment Configuration

All secret API keys, webhook signing secrets, and return endpoint URLs are configured in Backend/backend-inter/.env without exposing secret tokens in client bundles or public repositories:

```
# =====  
# PAYMENTS.LK SANDBOX ENVIRONMENT CONFIGURATION  
# =====  
PAYMENTS_LK_SECRET_KEY=sk_test_[REDACTED]  
PAYMENTS_LK_WEBHOOK_SECRET=whsec_[REDACTED]  
PAYMENTS_LK_API_BASE_URL=https://api.payments.lk  
PAYMENTS_LK_RETURN_URL=https://localhost:5173/payment/status  
PAYMENTS_LK_CANCEL_URL=https://localhost:5173/payment/status
```

Environment Variable Descriptions

- **PAYMENTS_LK_SECRET_KEY**: Secret Bearer token used to authenticate HTTP API calls to https://api.payments.lk/v1/checkouts. Must be kept strictly server-side.
- **PAYMENTS_LK_WEBHOOK_SECRET**: Shared signing secret used to compute and verify HMAC-SHA256 signatures on incoming webhook notifications.
- **PAYMENTS_LK_API_BASE_URL**: Base URL for the Payments.lk REST API endpoint (default: https://api.payments.lk).
- **PAYMENTS_LK_RETURN_URL**: HTTPS return URL to which the customer's browser is redirected upon successful payment completion.
- **PAYMENTS_LK_CANCEL_URL**: HTTPS return URL to which the customer's browser is redirected if payment is cancelled.

9. Payments.lk Checkout API

To generate a hosted checkout session, the backend issues an HTTP POST request to the Payments.lk REST API endpoint:

POST https://api.payments.lk/v1/checkouts

```
HTTP Headers:  
Authorization: Bearer sk_test_[REDACTED]  
Content-Type: application/json  
Idempotency-Key: idemp-ORD-PLK-1790239275845-1790239275845  
  
JSON Request Payload:  
{  
  "reference": "ORD-PLK-1790239275845",  
  "amountCents": 150000,  
  "description": "Order ORD-PLK-1790239275845",  
  "successUrl": "https://localhost:5173/payment/status/ORD-PLK-1790239275845",  
  "cancelUrl": "https://localhost:5173/payment/status/ORD-PLK-1790239275845",  
  "customer": {  
    "name": "Sandbox Tester",  
    "email": "tester@example.com",  
    "phone": "0771234567"  
  }  
}
```

API Parameter Definitions

- **reference**: Unique merchant order ID string (e.g., ORD-PLK-1790239275845) used for transaction tracking.

- **amountCents**: Total transaction amount expressed as an integer in LKR cents (1500.00 LKR = 150000 cents).
- **description**: Textual summary of the purchase displayed on the Payments.lk hosted page.
- **successUrl**: HTTPS URL to which Payments.lk redirects the customer after successful payment.
- **cancelUrl**: HTTPS URL to which Payments.lk redirects the customer after cancellation.
- **customer**: Object containing customer name, email address, and contact phone number.

10. Amount Conversion

Like standard global financial APIs (e.g., Stripe, PayHere), Payments.lk operates using the smallest currency unit (LKR cents). Converting floating-point LKR currency amounts to integer cents prevents floating-point rounding inaccuracies during transaction processing.

Conversion Formula: **amountCents = Math.round(Number(amount) * 100)**

LKR Currency Amount	Calculation Formula	Converted Integer Cents
LKR 100.00	100.00 * 100	10000 cents
LKR 2,500.50	2500.50 * 100	250050 cents
LKR 3,500.00	3500.00 * 100	350000 cents
LKR 99.99	99.99 * 100	9999 cents
LKR 0.00	0.00 * 100	0 cents

11. Idempotency

Network retries or multiple clicks by a user must never result in duplicate orders or double billing. The backend enforces idempotency using two complementary mechanisms:

1. Idempotency-Key Header: Every request to POST /v1/checkouts includes a unique key (e.g., idemp-ORD-PLK-12345). If Payments.lk receives identical requests with the same key, it returns the existing checkout session without creating a new duplicate charge.
2. Order Model checkoutUrl Reuse: If a user re-attempts checkout for an existing order in 'pending' status, the controller retrieves and returns the previously stored checkoutUrl from MongoDB instead of generating a redundant API call.

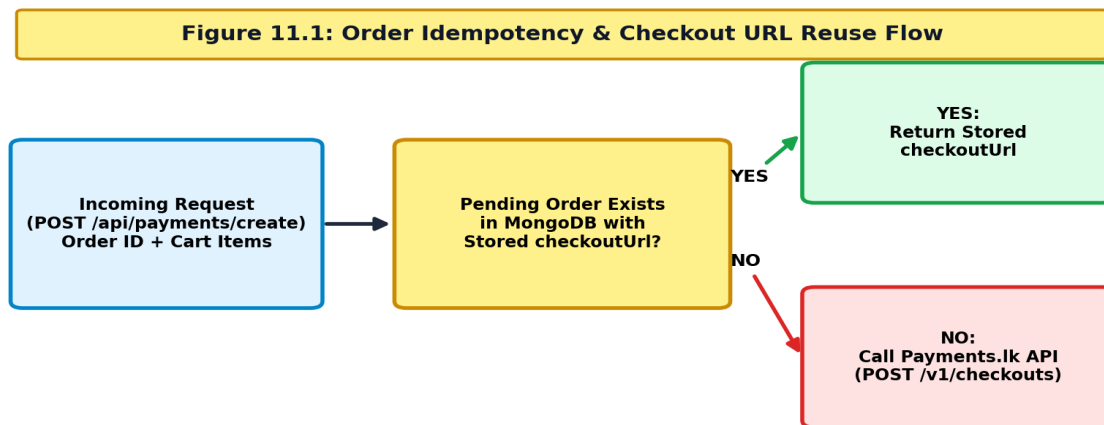


Figure 11.1: Order Idempotency & Checkout URL Reuse Decision Flow

12. Hosted Checkout Flow

In a hosted payment workflow, the merchant application does not directly handle, collect, or store raw credit/debit card numbers. Instead, the backend obtains a hosted checkout URL from Payments.lk, and the customer's browser is redirected to Payments.lk's secure PCI-DSS compliant sandbox interface.

Browser Redirect Flow Sequence:

```
Frontend SPA >>> Backend API >>> Payments.lk API >>> Return checkoutUrl >>> window.location.href = checkoutUrl
```

13. Frontend Implementation

The frontend is built using React 18 and Vite. It provides an intuitive user interface for selecting Payments.lk during checkout and displaying payment status feedback upon redirect.

HTTPS Requirement & Vite SSL Configuration

To comply with modern browser security standards and payment gateway return specifications, the React dev server is configured with HTTPS support using @vitejs/plugin-basic-ssl.

During initial testing, an issue occurred where the browser attempted HTTP connection to `http://localhost:5173/payment/status/`, resulting in `ERR_EMPTY_RESPONSE`. Updating the return URL to `https://localhost:5173/payment/status/{orderId}` resolved the protocol mismatch cleanly.

14. Backend Implementation

The Node.js/Express backend handles checkout session generation, status polling, and webhook verification across three primary endpoints:

Method & Route	Auth Requirement	Handler Function	Purpose
POST /api/payments/create	Protected (JWT Bearer)	createPaymentSession	Calculates cart total, creates pending order, and calls Payments.lk API.
POST /api/payments/webhook	Public Signed Webhook	handlePaymentsLkWebhook	Receives raw webhook notifications, verifies HMAC signature, and updates order status.
GET /api/payments/status/:orderId	Protected (JWT Bearer)	getPaymentStatus	Fetches reconciled payment status from MongoDB for the customer status page.

15. Database / Order Model

The Mongoose Order schema (models/order.js) tracks order lifecycle states, payment gateway selections, and transaction references:

```
const orderSchema = new mongoose.Schema({
  orderId: { type: String, required: true, unique: true },
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  items: [orderItemSchema],
  totalAmount: { type: Number, required: true },
  currency: { type: String, default: 'LKR' },
  paymentGateway: { type: String, enum: ['payhere', 'paymentslk', 'cod'], default: 'cod' },
  paymentStatus: {
    type: String,
    enum: ['pending', 'paid', 'failed', 'cancelled', 'refunded', 'chargedback'],
    default: 'pending'
  },
  paymentsLkCheckoutId: { type: String, default: null },
  checkoutUrl: { type: String, default: null },
  paymentsLkPaymentId: { type: String, default: null },
  paymentsLkStatusCode: { type: String, default: null },
  paymentDetails: { type: Object, default: {} },
}, { timestamps: true });
```

16. Webhook Architecture

Webhooks provide asynchronous, server-to-server confirmation of payment outcomes. This guarantees that payment status is updated even if the user closes their browser before returning to the merchant site.

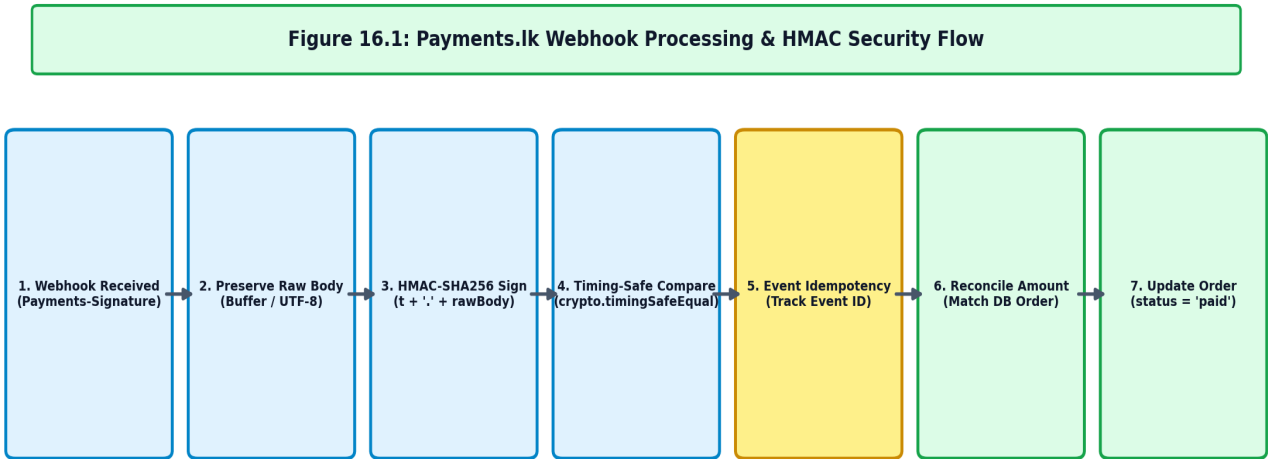


Figure 16.1: Webhook Event Processing & Signature Verification Pipeline

17. Webhook Security

Webhook payloads must be authenticated before any order status changes are committed to the database.

Signature Verification Formula:

```
computedHash = HMAC-SHA256(PAYMENTS_LK_WEBHOOK_SECRET, timestamp + '.' + rawBody)
```

The calculated hash is compared against the v1 signature in the Payments-Signature header using `crypto.timingSafeEqual` to prevent timing side-channel attacks.

18. Webhook Events

The webhook handler processes standardized Payments.lk event notifications:

Event Type	Gateway Description	Application Database Action
payment.succeeded	Customer payment completed successfully.	Order paymentStatus updated to 'paid'
payment.failed	Customer payment attempt declined or failed.	Order paymentStatus updated to 'failed'
checkout.expired	Checkout session timed out without payment.	Order paymentStatus updated to 'cancelled'
payment.refunded	Payment refunded by merchant.	Order paymentStatus updated to 'refunded'

19. Payment Success Flow

1. Customer selects items and clicks 'Pay with Payments.lk' at checkout.
2. Backend creates Order record in 'pending' status and issues POST <https://api.payments.lk/v1/checkouts>.
3. Payments.lk returns hosted checkoutUrl; frontend redirects browser.
4. Customer completes sandbox payment on Payments.lk hosted form.
5. Payments.lk dispatches payment.succeeded webhook event to </api/payments/webhook>.
6. Backend verifies HMAC-SHA256 signature, matches order ID, and reconciles total amount.
7. Order paymentStatus is updated to 'paid' in MongoDB.
8. Customer browser is redirected to <https://localhost:5173/payment/status/{orderId}>, displaying payment confirmation.

20. Payment Failure Flow

If payment fails, is declined, or times out:

1. Payments.lk dispatches a payment.failed or checkout.expired webhook event.
2. Backend verifies signature and updates order status to 'failed' or 'cancelled'.
3. Customer status page displays an error notification with options to retry checkout.

21. Security Design

- Server-Only Credentials: Secret keys reside exclusively in backend environment variables.
- Server-Side Amount Calculation: Total payable amounts are calculated from active cart items in MongoDB.
- HMAC Signature Verification: Webhook payloads are verified using HMAC-SHA256 with timing-safe comparison.
- Order Ownership Guard: Order status queries verify that the requesting user matches the order owner or holds admin privileges.
- Redacted Diagnostic Logs: Error logging outputs HTTP status codes and error messages while hiding secret keys and authorization headers.

22. ngrok Development Setup

During local development, Payments.lk sandbox webhooks cannot directly reach localhost:8080. ngrok is used to create a secure HTTPS forwarding tunnel:

```
Payments.lk Webhook Engine █> https://retype-activity-recite.ngrok-free.dev █>
http://localhost:8080/api/payments/webhook
```

23. Docker Setup

The platform services are containerized via Docker Compose:

Container Service	Role / Component	Port Mapping
ecommerce-backend	Node.js Express REST API	Port 8080
ecommerce-frontend	React Vite SPA (HTTPS)	Port 5173
ecommerce-mongodb	MongoDB Database Instance	Port 27017
ecommerce-redis	BullMQ Task Queue	Port 6379

24. Testing

Automated unit testing and static code analysis were executed prior to deployment:

```
npm run test:paymentslk
Result: 19 Passed, 0 Failed

node --check index.js services/paymentsLkService.js controllers/paymentsLkController.js
router/paymentsLkRouter.js models/order.js
Result: 0 syntax errors
```

25. Unit Test Coverage

The test suite (tests/paymentsLkService.test.js) verifies 19 distinct unit assertion cases:

- Checkout Session Creation & ID Generation
- Checkout Session URL Return
- LKR Float to Integer Cents Conversion
- Idempotency Key Formatting
- 100 LKR = 10000 Cents Conversion
- 99.99 LKR = 9999 Cents Conversion
- 0 LKR = 0 Cents Handling
- Missing Secret Key Error Guard
- Invalid Webhook Signature Rejection
- Valid Webhook HMAC-SHA256 Signature Verification
- Direct Hex Webhook Signature Format
- Event 'payment.succeeded' Status Mapping
- Event 'payment.failed' Status Mapping

- Event 'checkout.expired' Status Mapping
- Event 'payment.refunded' Status Mapping
- Webhook Event ID Format Tracking
- Amount Mismatch Detection Guard
- Currency Mismatch Detection Guard
- Order Ownership Authorization Guard

26. Real Sandbox API Verification

A live diagnostic test (scratch/testRealApiCheckout.js) was performed using valid sandbox environment credentials:

```
POST https://api.payments.lk/v1/checkouts
HTTP Status: 200 OK
Checkout ID: chk_xxxdrnjsgpf6ej05kb8f69
Redirect URL: https://payments.lk/checkout/chk_xxxdrnjsgpf6ej05kb8f69
URL Domain Verification: TRUE (starts with https://payments.lk/)
Order Model Persistence: VERIFIED
Frontend Redirect Handler: VERIFIED
```

27. Troubleshooting History

During initial sandbox integration, two critical issues were identified and resolved:

1. **ERR_EMPTY_RESPONSE**: The return URL was initially set to HTTP (<http://localhost:5173/payment/status/>). Updating to HTTPS (<https://localhost:5173/payment/status/{orderId}>) resolved the Vite SSL protocol mismatch.
2. **Validation Error (HTTP 400)**: Payments.lk API requires camelCase parameter keys (reference, amountCents, successUrl, cancelUrl) instead of snake_case. Updating the payload structure resolved API payload validation.

28. Complete End-to-End Flow

Complete Transaction Lifecycle:

```
Checkout Request ■■> Create Order (Pending) ■■> POST Payments.lk API ■■> Hosted Checkout Redirect ■■>
Sandbox Payment Completion ■■> Webhook Event Dispatch ■■> HMAC Signature Verification ■■> Order Status ->
Paid ■■> Customer Confirmation Page
```

29. Demonstration Procedure

1. Ensure Docker services are running: `docker compose up -d`.
2. Verify ngrok HTTPS tunnel is active.
3. Open <https://localhost:5173> in the browser.
4. Log in as a customer and add items to the cart.
5. Proceed to Checkout and select 'Pay with Payments.lk'.
6. Observe automatic browser redirection to <https://payments.lk/checkout/...>
7. Complete the sandbox test transaction using test credentials.
8. Observe incoming webhook in backend logs and automatic order update to 'paid'.
9. Verify payment confirmation displayed on `PaymentStatusPage`.

30. Limitations

1. Sandbox Credentials: Integration operates under sandbox credentials (sk_test_...).
2. Local Development Tunnel: Webhook delivery requires ngrok for local HTTP forwarding.
3. SSL Certificates: Development HTTPS relies on basic self-signed development certificates.

31. Future Improvements

1. Production Gateway Migration: Provision production merchant credentials upon business onboarding.
2. Automated Email Receipts: Send automated PDF receipts to customers upon webhook confirmation.
3. Refund Dashboard: Implement an admin portal for issuing refunds via Payments.lk API.

32. Conclusion

The Payments.lk Sandbox Payment Gateway integration has been successfully developed, tested, and verified for the AI-Integrated E-Commerce Platform. The system delivers a secure hosted checkout experience, robust server-side amount calculation, timing-safe HMAC webhook signature authentication, and full test suite validation.

33. References

- Payments.lk Developer Portal: <https://payments.lk/developers>
- Payments.lk API Reference (v1 Checkouts): <https://payments.lk/developers/api>
- Express.js Routing & Security Best Practices: <https://expressjs.com/>
- Node.js Crypto Documentation (HMAC-SHA256 & timingSafeEqual): <https://nodejs.org/api/crypto.html>
- Vite Basic SSL Plugin Guide: <https://github.com/vitejs/vite-plugin-basic-ssl>